

Abschlussprojekt "Diskrete Simulation" SS2025

Modul: Diskrete Simulation

Studiengang: Angewandte Informatik

Professor: Prof. T. Wiedemann

Author: Alexander Schulz (alexander.schulz2@stud.htw-dresden.de)

Inhaltsübersicht

1. Einleitung	3
2. Auswahl der Simulationsumgebung	3
2.1. Vergleichene Crates	3
2.1.1. Nexosim	3
2.1.2. Desim	3
2.1.3. SimRS	4
2.2. Begründung für die Auswahl von Nexosim	4
2.3. NeXosim Framework: Kernkomponenten und Paradigmen	4
2.3.1. Kernkomponenten von NeXosim	4
2.3.2. Zentrale Paradigmen	5
2.3.3. Aktormodell	5
2.3.4. Component-Flow Architecture	5
2.3.5. Diskrete Ereignissimulation	5
2.3.6. Parallele Ausführung	5
3. Systemanalyse und Abstraktion	5
3.1. Problemstellung	5
3.2. Abstraktion des Systems	6
4. Konzeption	6
5. Implementierung	6
5.1. Grundlegende Struktur	6
5.2. Performance-Tests	6
5.2.1. Simulation mit Debug-Ausgaben (Development-Build)	7
5.2.2. Simulation ohne Debug-Ausgaben (Development-Build)	7
5.2.3. Simulation mit Debug-Ausgaben (Release-Build)	7
5.2.4. Simulation ohne Debug-Ausgaben (Release-Build)	7
6. Herausforderungen und Erfahrungen	8
6.1. Herausforderungen bei der Implementierung	8
6.2. Bewertung und Erfahrungen	8
6.2.1. Modellierungskomfort	9
6.2.2. Flexibilität	9
6.2.3. Funktionalität	9
6.2.4. Performance	9
6.2.5. Eignung für Studierende	9
7. Fazit und Ausblick	9
8. Anhang	9
8.1. Logging der Warteschlange	9

1. Einleitung

In diesem Abschlussprojekt für das Modul "Diskrete Simulation" wurde eine Simulation eines Schwimmbadsystems implementiert. Das Projekt demonstriert die Anwendung von Konzepten der diskreten Simulation zur Modellierung und Analyse eines Systems mit beschränkter Kapazität und Steuerungselementen. Ziel des Projektes ist es, die Eignung und Performance von Nexosim, einem Rust-basierten Framework für diskrete Ereignissimulationen, zu evaluieren.

2. Auswahl der Simulationsumgebung

Bei der Auswahl einer geeigneten Simulationsumgebung in Rust wurden mehrere Crates verglichen und bewertet. Die Bewertungskriterien umfassten:

- Aktive Weiterentwicklung des Crates
- Qualität der Dokumentation
- Performance-Eigenschaften
- Verbreitungsgrad (Anzahl der Downloads auf crates.io)

2.1. Vergleichene Crates

2.1.1. Nexosim

nexosim v0.3.3 A high performance asynchronous compute framework for system simulation. Repository	↓ All-Time: 3,099 ↓ Recent: 1,535 🔄 Updated: about 1 month ago
asynchronix v0.2.4 [Asynchronix is now NeXosim] A high performance asynchronous compute framework for system simulation. Repository	↓ All-Time: 16,951 ↓ Recent: 4,348 🔄 Updated: 7 months ago

2.1.2. Desim

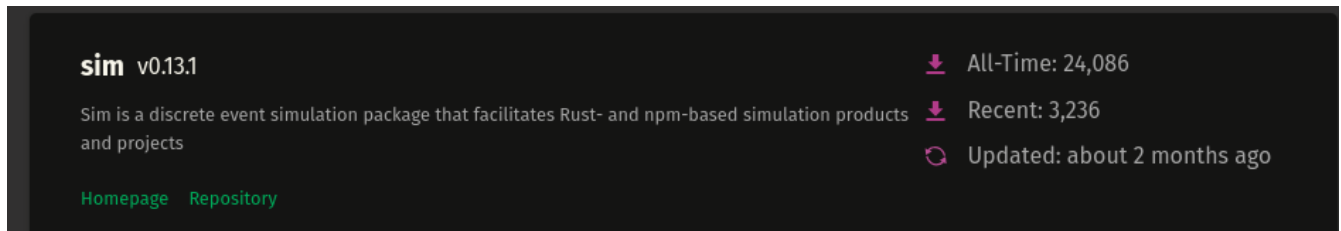
[Link: desim - Rust](<https://docs.rs/desim/latest/desim/>)

desim v0.4.0 A discrete-time events simulation framework inspired by Simpy Repository	↓ All-Time: 5,264 ↓ Recent: 691 🔄 Updated: over 1 year ago
--	--

Hinweis: Nexosim hieß vorher Asynchronix und wird unter diesem Namen noch in vielen Projekten verwendet, was die hohe Anzahl an Downloads erklärt.

2.1.3. SimRS

[Link: SimRS · SimRS](https://simrs.com/)



2.2. Begründung für die Auswahl von Nexosim

Nach Abwägung der verschiedenen Optionen fiel die Wahl auf Nexosim. Ausschlaggebend waren:

- Umfangreiche und verständliche Dokumentation (Beispiele vorhanden)
- Aktive Weiterentwicklung mit regelmäßigen Updates
- Vorteilhafte Performance-Eigenschaften
- Wird bereits bei anderen Projekten verwendet (Downloads auf crates.io)

2.3. NeXosim Framework: Kernkomponenten und Paradigmen

(Übersetzung und Zusammenfassung der Projekt-README)

NeXosim ist ein hochoptimiertes Framework für diskrete Ereignissimulationen in Rust, das durch seinen komponentenorientierten Ansatz und eine asynchrone Implementierung besticht. Das Framework unterstützt sowohl einfache Simulationen als auch komplexe Simulationsszenarien mit zeitgesteuerten Zustandsautomaten.

2.3.1. Kernkomponenten von NeXosim

1. **Modelle** (Models): Die grundlegenden Bausteine einer Simulation, die als isolierte Entitäten mit definierten Ein- und Ausgängen fungieren und durch Ereignisse kommunizieren.
2. **Mailboxen** (Mailboxes): Empfangen Nachrichten für ein bestimmtes Modell und fungieren als Schnittstelle für den asynchronen Nachrichtenaustausch.
3. **Ports**: Typisierte Ein- und Ausgänge für Modelle:
 - **Output<T>**: Sendet Ereignisse an verbundene Modelle
 - **EventSlot<T>**: Empfängt Ereignisse für weitere Verarbeitung
4. **Kontext** (Context): Ermöglicht Modellen, Ereignisse zu planen und auf Ressourcen zuzugreifen.
5. **Scheduler**: Verwaltet die Ereignisreihenfolge und den zeitlichen Fortschritt der Simulation.

2.3.2. Zentrale Paradigmen

2.3.3. Aktormodell

NeXosim implementiert das Aktormodell, bei dem jedes Simulationsmodell als eigenständiger Akteur fungiert, der seinen internen Zustand verwaltet und über Nachrichtenaustausch mit anderen Akteuren kommuniziert.

2.3.4. Component-Flow Architecture

Die Architektur ähnelt dem Flow-Based Programming (FBP), wobei Modelle durch definierte Verbindungen miteinander kommunizieren, was eine klare Trennung der Komponenten und eine übersichtliche Systemstruktur ermöglicht.

2.3.5. Diskrete Ereignissimulation

Die Simulation schreitet in diskreten Zeitschritten voran. Der Scheduler wählt nach Abschluss der Berechnungen für den aktuellen Zeitpunkt den nächsten Zeitpunkt (Next Event Increment), zu dem Ereignisse geplant sind.

2.3.6. Parallele Ausführung

NeXosim nutzt Rusts Unterstützung für Multithreading und asynchrone Programmierung, um Simulationsmodelle effizient parallel auszuführen, wobei die erforderliche Synchronisierung transparent vom Simulator gehandhabt wird.

Weitere Informationen und Dokumentation finden Sie unter:

- [GitHub Repository](<https://github.com/asynchronics/nexosim>)
- [API-Dokumentation](<https://docs.rs/nexosim>)
- [Crates.io Seite](<https://crates.io/crates/nexosim>)

Die modulare Struktur und die klaren Schnittstellen machen NeXosim zu einer leistungsstarken Wahl für die Modellierung komplexer Systeme wie das in diesem Projekt implementierte Schwimmbadsystem.

3. Systemanalyse und Abstraktion

3.1. Problemstellung

Die Aufgabe bestand in der Modellierung eines Schwimmbadsystems, bei dem die maximale Belegung überwacht und gesteuert werden muss. Personen betreten und verlassen das Schwimmbad, wobei die Gesamtkapazität nicht überschritten werden darf.

3.2. Abstraktion des Systems

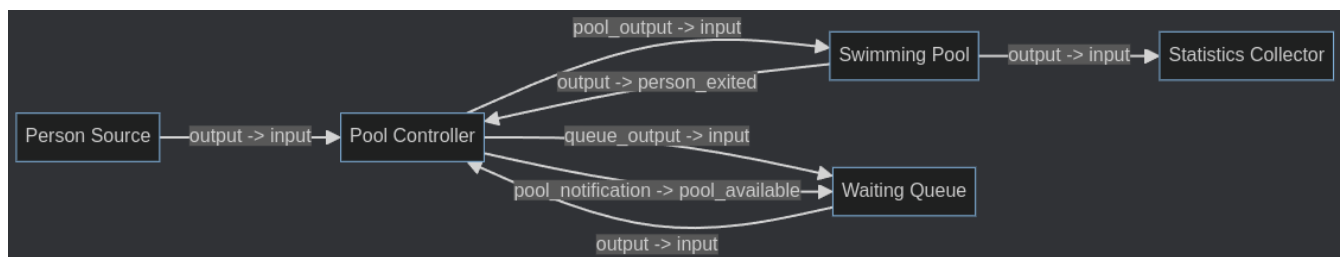
Das Schwimmbadsystem wurde auf folgende Kernelemente abstrahiert:

- Eingangsbereich (PersonSource)
- Warteschlange, falls das Schwimmbad belegt ist (WaitingQueue)
- Schwimmbereich mit begrenzter Kapazität (SwimmingPool)
- Steuerungseinheit (PoolController) zur Überwachung der Belegung

Diese Abstraktion ermöglicht eine übersichtliche Modellierung des Systems und erlaubt die Simulation verschiedener Belegungsszenarien.

4. Konzeption

Die Konzeption des Simulationsmodells folgte einem strukturierten Ansatz. Zunächst wurden Basismodelle für die verschiedenen Komponenten des Systems entwickelt. Anschließend wurde ein Controller eingeführt, der die maximale Belegung im Schwimmbad überwacht und steuert.



Die Modellstruktur zeigt die Interaktion zwischen den verschiedenen Komponenten und verdeutlicht den Fluss der Personen durch das System. Der Controller übernimmt dabei eine zentrale Rolle bei der Steuerung des Zugangs zum Schwimmbad.

5. Implementierung

Alle Simulationen verwenden einen festen Seed, um reproduzierbare Ergebnisse zu gewährleisten, was besonders für Tests und Performance-Evaluationen wichtig ist.

5.1. Grundlegende Struktur

Die Implementierung folgt dem Model-Connect-Event-Prinzip:

1. **Instanz (Instance):** Repräsentiert ein Modell in der Simulation
2. **Connect:** Verbindet verschiedene Modelle miteinander
3. **Events:** Werden von Instanzen ausgelöst und an die verbundenen Mailboxen weitergeleitet

5.2. Performance-Tests

Zur Evaluierung der Performance wurden verschiedene Testläufe durchgeführt:

5.2.1. Simulation mit Debug-Ausgaben (Development-Build)

```
Wall clock execution time: 82.163478ms
Swimming Pool Simulation Statistics:
- Total participants: 468
- Average swim time: 59.94 minutes
- Average wait time: 0.13 minutes
- Maximum wait time: 8.27 minutes
Simulation completed successfully
```

5.2.2. Simulation ohne Debug-Ausgaben (Development-Build)

```
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
Running `target/debug/ds-smd-simulation`
Loaded configuration from config.toml
Starting simulation for 2400 minutes...
Wall clock execution time: 58.384447ms
Swimming Pool Simulation Statistics:
- Total participants: 468
- Average swim time: 59.94 minutes
- Average wait time: 0.13 minutes
- Maximum wait time: 8.27 minutes
Simulation completed successfully
```

5.2.3. Simulation mit Debug-Ausgaben (Release-Build)

```
Wall clock execution time: 27.264038ms
Swimming Pool Simulation Statistics:
- Total participants: 468
- Average swim time: 59.94 minutes
- Average wait time: 0.13 minutes
- Maximum wait time: 8.27 minutes
Simulation completed successfully
```

5.2.4. Simulation ohne Debug-Ausgaben (Release-Build)

```
Finished `release` profile [optimized] target(s) in 0.07s
Running `target/release/ds-smd-simulation`
Loaded configuration from config.toml
Starting simulation for 2400 minutes...
Wall clock execution time: 7.226317ms
Swimming Pool Simulation Statistics:
- Total participants: 468
- Average swim time: 59.94 minutes
- Average wait time: 0.13 minutes
- Maximum wait time: 8.27 minutes
```

Die Performance-Tests zeigen, dass Nexosim in der Lage ist, auch komplexe Simulationsszenarien effizient zu verarbeiten. Die Release-Builds bieten eine signifikante Performance-Steigerung im Vergleich zu den Development-Builds, insbesondere bei deaktivierten Debug-Ausgaben mit nur **7.22 ms**.

Für weitere Tests können die Simulationen mit angepassten Parametern (`config.toml`) durchgeführt werden.

6. Herausforderungen und Erfahrungen

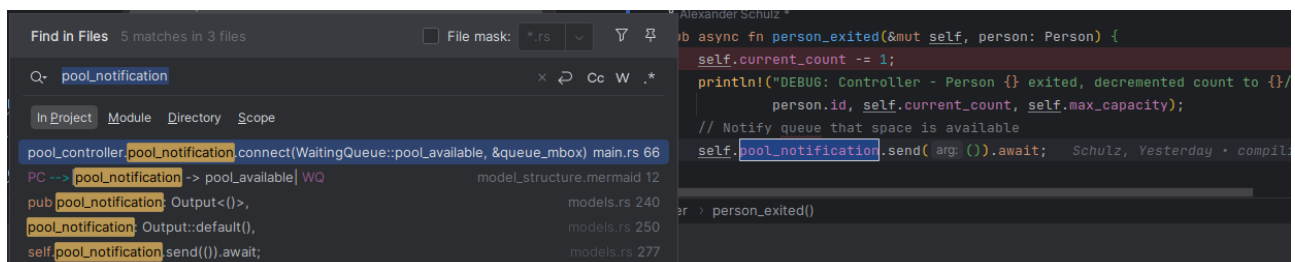
6.1. Herausforderungen bei der Implementierung

Die Arbeit mit Nexosim wies einige Herausforderungen auf:

1. **Fehlermeldungen:** Die Fehlermeldungen waren teilweise schwer verständlich (für unerfahrende Rust-Entwickler), wie das folgende Beispiel zeigt:

```
70 |     swimming_pool.output.connect(PoolController::person_exited,
    |                                ----- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
    |                                `for`<'a> InputFn<'a, _, Person, _>` is not implemented for fn item `for`<'a> fn(&'a
    |                                mut PoolController) -> impl Future<Output = ()> {PoolController::person_exited}`
```

2. **Modell-Komplexität:** Die Modelle können schnell unübersichtlich werden, auch wenn eine gute Strukturierung prinzipiell möglich ist.
3. **Event-Nachverfolgung:** Da Events in Modellen definiert und später in der Main-Funktion verbunden werden, erfordert das Nachverfolgen des Programflusses ein Springen zwischen verschiedenen Codeteilen.



4. **Monitoring:** Die Integration von Monitoring-Funktionalitäten ist nicht trivial, aber durchaus machbar unter Verwendung einer `EventQueue`.

6.2. Bewertung und Erfahrungen

6.2.1. Modellierungskomfort

Bei vorhandenen Rust-Kenntnissen ist die Arbeit mit Nexosim vergleichsweise einfach, erfordert jedoch eine gewisse Einarbeitungszeit. Die Lernkurve ist vorhanden, aber zu bewältigen.

6.2.2. Flexibilität

Das Framework bietet eine hohe Flexibilität bei der Modellierung verschiedener Systeme. Die aktorbasierte Architektur ermöglicht eine klare Trennung der Komponenten und einen übersichtlichen Nachrichtenaustausch.

6.2.3. Funktionalität

Die Funktionalität von Nexosim ist für die Anforderungen einer diskreten Simulation gut geeignet. Es bietet alle notwendigen Funktionen für die Modellierung, Simulation. Für die Auswertung der Ergebnisse müssen jedoch zusätzliche Funktionen geschrieben werden.

6.2.4. Performance

Die Performance ist sehr gut und eignet sich vorraussichtlich auch für komplexere Simulationsmodelle.

6.2.5. Eignung für Studierende

Nexosim ist für Studierende geeignet, wenn grundlegende Kenntnisse in Rust vorhanden sind. Die Dokumentation und die vorhandenen Beispiele erleichtern den Einstieg.

7. Fazit und Ausblick

Die Implementierung des Schwimmbadsystems mit Nexosim hat gezeigt, dass das Framework gut für die diskrete Simulation komplexer Systeme geeignet ist. Die Performance ist überzeugend und die Modellierungsmöglichkeiten sind vielseitig.

Für zukünftige Projekte könnte eine verbesserte Visualisierung der Simulationsergebnisse sowie eine vereinfachte Möglichkeit zum Monitoring von Zustandsänderungen sinnvoll sein.

8. Anhang

8.1. Logging der Warteschlange

```
DEBUG: Controller - Sending person 0 to pool. Current count: 1/5
DEBUG: Controller - Sending person 1 to pool. Current count: 2/5
DEBUG: Controller - Sending person 2 to pool. Current count: 3/5
DEBUG: Controller - Sending person 3 to pool. Current count: 4/5
DEBUG: Controller - Sending person 4 to pool. Current count: 5/5
DEBUG: Controller - Pool full, sending person 5 to queue. Current count: 5/5
```

```
DEBUG: WaitingQueue - Person 5 added to queue. Queue length: 1
DEBUG: Controller - Pool full, sending person 6 to queue. Current count: 5/5
DEBUG: WaitingQueue - Person 6 added to queue. Queue length: 2
DEBUG: Controller - Pool full, sending person 7 to queue. Current count: 5/5
DEBUG: WaitingQueue - Person 7 added to queue. Queue length: 3
DEBUG: Controller - Pool full, sending person 8 to queue. Current count: 5/5
DEBUG: WaitingQueue - Person 8 added to queue. Queue length: 4
DEBUG: Controller - Pool full, sending person 9 to queue. Current count: 5/5
DEBUG: WaitingQueue - Person 9 added to queue. Queue length: 5
DEBUG: Controller - Pool full, sending person 10 to queue. Current count: 5/5
DEBUG: WaitingQueue - Person 10 added to queue. Queue length: 6
DEBUG: Controller - Person 4 exited, decremented count to 4/5
DEBUG: WaitingQueue - Person 5 left queue. Queue length: 5
DEBUG: Controller - Sending person 5 to pool. Current count: 5/5
DEBUG: Controller - Pool full, sending person 11 to queue. Current count: 5/5
```